NEURALSPARK

Project Report Executive Summary & Plain English Presentation Guide

A Jargon-Free Companion Guide to Understand, Present, and Defend the Thesis

Project Team Members:

Malay Singh (2303031249003) **Prashant Nagre** (2303031240687)

Shreyansh Negi (2303031241262) **Nitesh Sukhwal** (2303031240866)

Project Guide: **Prof. RASNA PATEL** (AI/DS Department, PIET)

Department of Artificial Intelligence and Data Sciences

Parul Institute of Engineering and Technology, Parul University, Vadodara

Academic Year: 2025–2026

Purpose of this Document: This companion guide condenses our full 71-page academic thesis into clear, everyday English. Use it to rehearse your project presentation, review key technical decisions, explain the evolution from the initial 34-page draft to the 71-page final report, and confidently answer viva defense questions.

---

1 The 30-Second Elevator Pitch (How to Explain NeuralSpark)

> *"NeuralSpark is an all-in-one, AI-powered e-learning platform built specifically for competitive engineering exams like GATE. It combines video lectures, theory notes, previous year questions, and community doubt forums into one place. Most importantly, it features an intelligent AI Grader that can evaluate handwritten mathematical proofs and assignments in under 2.5 seconds with 92.8% human-level accuracy, and it can run completely offline in rural classrooms without internet."*

2 Report Evolution: How the 34-Page Draft Expanded to 71 Pages

To meet full academic and university publishing standards, the initial 34-page draft was comprehensively expanded into a rigorous 71-page thesis across six key areas:

1. **Front Matter & University Compliance (Pages 1–8):**
   - Created a strict 1-page Title/Cover layout (Red bold title, all 4 students, guide, and Parul University crest).
   - Integrated the official Certificate page with Subject Code `203105400`, Group `PIET-CSE-111`, and signatures for Guide, HOD, Project Coordinator, and External Examiner.
   - Added Acknowledgements, Abstract, Table of Contents, List of Tables, List of Figures, and List of Algorithms.

2. **Chapter 1: Introduction (Expanded to 14 Pages):**
    - Added historical evolution of automated essay scoring (Page's PEG, LSA, e-rater to modern LLMs).
    - Documented the 6 root causes of traditional LMS failure (faculty burnout, grading bottleneck, software fragmentation, rural connectivity divide).
    - Formulated complete curriculum scope across the 8 GATE CS/DA subjects and 7 technical objectives.
3. **Chapter 2: Literature Survey (Expanded to 16 Pages):**
    - Expanded to 15 in-depth individual research paper evaluations with mathematical equations for self-attention, masked language modeling, RoPE, and Quadratic Weighted Kappa (κ).
    - Redesigned the Literature Survey Comparison Matrix (Table 2.1) with full-width columns to prevent syllable squishing.
4. **Chapter 3: Methodology & Architecture (Expanded to 22 Pages):**
    - Added Raspberry Pi 4 Model B specs, standalone Wi-Fi Access Point mode, and microcontroller comparisons (Arduino vs ESP8266 vs ESP32).
    - Added educational IoT sensor modules (Voice V3, gesture sensor, I2C LCD, 10.1" HDMI touch display).
    - Added comprehensive Cloud Provider Matrix (AWS vs GCP vs Azure: Lambda, S3, RDS, SageMaker).
    - Added deep AI mathematics: MLP backprop derivations, CNN convolutions, LSTM gates, and PPO Reinforcement Learning.
    - Added complete field schemas for all 14 Django ORM models and 4 formal LaTeX algorithms.
5. **Chapter 4: Implementation & Results (Expanded to 16 Pages):**
    - Captured and embedded 8 real, high-resolution application screenshots from the active Next.js web app.
    - Added a 15-case Quality Assurance Verification Test Suite (Table 4.2).
    - Documented benchmark results across 500 student submissions ($\kappa = 0.887$ human-parity agreement).
    - Added 3 qualitative case studies with step-by-step mathematical derivation checking.
6. **Chapter 5 & Bibliography (Pages 67–71):**
    - Formulated 4 future research roadmaps (On-device INT4 SLMs, vernacular Indian languages, GNN knowledge graphs).
    - Expanded to 35+ complete IEEE-formatted academic citations.

Metric / Component	Initial Draft	Final Comprehensive Report
Total Page Count	34 Pages	**71 Pages (High-Density Thesis)**
Cover Page Layout	Overflown to 2 pages	**Strictly 1 Page (Perfect Fit)**
Figures & Screenshots	2 General Diagrams	**13 High-Resolution Figures & UI Screens**
Database Models	4 Summarized	**All 14 Django ORM Models Detailed**
Literature Papers	5 Brief Summaries	**15 Deep Multi-Page Evaluations**
LaTeX Algorithms	1 Generic	**4 Formal LaTeX Algorithms with Pseudo-code**
QA Test Cases	None	**15 Comprehensive Verification Logs**
Qualitative Case Studies	None	**3 In-depth Step-by-Step Derivation Cases**

3 The Real-World Problem: Why Did We Build This?

1. **The Grading Scalability Bottleneck (Teacher Burnout):** When a professor teaches a class of 500 students, grading one weekly subjective assignment takes over 30 to 40 hours of manual labor. Because of this fatigue, students often wait 1 to 2 weeks to get feedback, which breaks the learning cycle.

2. **No Way to Grade Handwritten Work with Existing Tools:** Current online platforms (like Google Forms or LeetCode) only check multiple-choice bubbles or automated coding test cases. They cannot read, understand, or grade scanned handwritten math steps, diagrams, or algorithmic derivations.

3. **Software Fragmentation:** Students are forced to switch between 5 different disconnected apps: YouTube for videos, Stack Overflow for doubts, Google Drive for old exam papers, and Google Forms for quizzes.

4. **The Rural Internet Divide:** Most modern AI tools crash if there is no high-speed internet. Rural and off-grid schools cannot use cloud-only platforms.

4 The 5 Core Features (What Did We Build?)

1. **Standardized 8-Subject Curriculum:** Covers the complete syllabus across 8 core domains: *Linear Algebra, Calculus, Probability & Statistics, Python Programming, Data Structures & Algorithms, Machine Learning, Artificial Intelligence, and General Aptitude*.

2. **Dual-Engine AI Grader:** A hybrid assessment system:

   - **Mode A (Cloud Multimodal Vision Grader):** Reads uploaded PDF/image assignments using **Llama 3.2 Vision**, verifies derivations against a rubric, and gives step-by-step mark breakdowns in under 2.5 seconds.
   - **Mode B (Local Transformer Grader):** Runs a locally fine-tuned Transformer model (BERT/RoBERTa) on the server's CPU to grade text answers in **sub-120 milliseconds** with zero cloud cost and 100% privacy.

3. **Interactive PYQ Practice Engine:** Previous year questions with 4-choice options, instant grading, and verified step-by-step mathematical explanations.

4. **Diagnostic Quizzes & Test Series:** Timed multi-question assessments with animated score counters and remedial feedback.

5. **Gamified Community Doubt Forum:** A peer-to-peer Q&A board where students help each other and earn reputation points when their answers are upvoted.

6. **Offline Classroom Suite (`run_offline.bat`):** A one-click automated batch launcher that boots the local database, backend API, frontend web app, and offline voice tutoring without requiring any internet.

5 How the Dual-Engine AI Grader Works (Simple Explanation)

Feature	Mode A: Cloud Vision Grader	Mode B: Local Transformer Grader
AI Model	Llama 3.2 Vision (via Groq API)	Fine-tuned RoBERTa / BERT (PyTorch)
Input Type	Scanned PDF assignments & Photos	Raw text answers & typed derivations
Speed	$\sim$ 2.15 seconds	$\sim$ 115 milliseconds (0.11s)
Accuracy	**92.8%** ($\kappa = 0.887$)	**88.6%** ($\kappa = 0.841$)
Feedback Type	Itemized rubric mark deductions	Numerical score interval prediction
Internet Required?	Yes	**No (100% Offline & Private)**

6 The Tech Stack Explained (Why We Chose Each Tool)

- **Backend: Django 6.0.2 + Django Ninja (Python)**
  - *Why Django Ninja?* Traditional Django REST Framework (DRF) is slow. Django Ninja uses Python async handlers and Pydantic validation, making our API up to 4$\times$ **faster** with sub-35ms response times.
- **Frontend: Next.js 16 + React 19 + TypeScript + Tailwind CSS 4**
  - *Why Next.js App Router?* Renders static pages on the server for instant page loads, while streaming interactive quiz components to the browser.
  - *Why Framer Motion 12?* Adds spring-physics animations to buttons, cards, and theme toggles for a smooth, premium user experience.
- **Database: SQLite (Local) / PostgreSQL (Production)**
  - 14 relational tables managing Users, Categories, PYQs, Questions, Answers, Comments, Quizzes, Articles, Lectures, and Inquiries.
- **Offline Voice AI: Vosk (STT) + Piper (TTS)**

- Allows students in rural classrooms to speak their doubts into a microphone and hear spoken answers without an internet connection.

7 Top 10 Viva Defense Questions and Exact Model Answers

Q1: What is the main novelty of NeuralSpark compared to existing LMS like Moodle or Canvas?
Answer: "Moodle and Canvas are passive content repositories that require 100% manual grading by teachers. NeuralSpark introduces an active, dual-engine AI grading pipeline that parses scanned handwritten student work against rubrics in 2.5 seconds, while also providing an all-in-one ecosystem with offline classroom capabilities."

Q2: How do you measure if the AI grading is accurate?
Answer: "We evaluated 500 student submissions graded by 3 expert faculty members. We used **Quadratic Weighted Kappa (κ)**, which is the gold standard for grading agreement. Our Multimodal Vision engine achieved $\kappa = 0.887$ (near-perfect agreement) and an F_1-score of 0.924."

Q3: Why do you have two grading engines instead of just one?
Answer: "Because of the trade-off between power and privacy/cost. Mode A (Cloud Llama 3.2 Vision) provides deep visual step parsing for complex exams, while Mode B (Local Transformer) runs locally on the CPU in under 120ms for instant practice and zero-cost offline classroom grading."

Q4: What happens if a student uploads a PDF assignment?
Answer: "The backend uses `PyPDF2` to extract text and `pdf2image` to render high-resolution 300 DPI page images. These images are Base64 encoded and sent to the multimodal Vision-LLM along with the rubric to perform visual mathematical derivation checking."

Q5: How does NeuralSpark run without internet?
Answer: "We created an automated batch launcher (`run_offline.bat`). It boots a local SQLite database, starts the Django Ninja API on port 8000, starts the Next.js server on port 3000, and uses local Vosk/Piper models for voice interaction on a local Wi-Fi LAN."

Q6: What database architecture did you design?
Answer: "We designed 14 normalized relational models in Django ORM with foreign keys and cascade delete rules, including User reputation tracking, subject categorization, PYQs, quizzes, and Q&A forum threads."

Q7: Why did you choose Next.js 16 and React 19 for the frontend?
Answer: "Next.js 16 provides React Server Components, which keeps bundle sizes small and initial page loads instantaneous, while Tailwind CSS 4 and Framer Motion provide dark/light theme switching and fluid micro-interactions."

Q8: How do you prevent AI hallucinations in grading?
Answer: "We enforce strict rubric-grounded system prompts and require the model to produce a structured JSON schema with itemized point deductions and line-by-line justifications rather than conversational text."

Q9: What are the latency benchmarks of your system?

Answer: "Standard API calls (quizzes, PYQs, theory notes) respond in 22ms to 34ms. Local Transformer grading takes 115ms. Cloud Multimodal grading takes 2.15 seconds."

Q10: What is your future roadmap for NeuralSpark?

Answer: "We plan to deploy 4-bit quantized Small Language Models (SLMs like Phi-3 or Llama-3.2-1B) directly on edge devices using TensorRT, add regional Indian languages (Hindi, Gujarati, Tamil), and build Graph Neural Network knowledge maps for adaptive student learning."

8 Key Benchmark Results at a Glance

Metric	Traditional Manual Grading	NeuralSpark Platform	Improvement Factor
Turnaround Latency	24 to 72 hours	$<$ **2.5 seconds**	$> 30,000\times$ **faster**
Grading Accuracy	94.0% (human baseline)	**92.8%** ($\kappa = 0.887$)	**Human-level parity**
Local Inference Speed	N/A	**115 milliseconds**	**Instant zero-latency**
API Response Time	180ms (Legacy DRF)	**28ms (Django Ninja)**	$6\times$ **faster**
Offline Support	None (Fails offline)	**Full Offline Suite**	**100% operational**

All materials, code, and PDF reports are available in the project workspace:

`https://github.com/malay336/NeuralSpark.git`